Banco Orbital

Documentación del proyecto

Un home banking completo, construido entre dos personas, andando en internet.

Octubre de 2026 · orbital.net.ar · Máximo Bertaina y Gonzalo Castellano

1. Qué es

Banco Orbital es un sistema de home banking funcionando en producción. No es una maqueta: tiene cuentas en pesos y en dólares, transferencias que salen y entran de otros bancos a través del Banco Central, tarjetas de débito y crédito con niveles, préstamos con evaluación de riesgo crediticio, plazos fijos, pago de servicios, cobros por QR y un asistente con inteligencia artificial que responde sobre la situación real de quien pregunta.

Lo usan usuarios reales con cuentas reales, la plata se mueve de verdad entre cuentas, y cada operación queda registrada en una auditoría de la que nadie puede escaparse.

Portal	https://app.orbital.net.ar
Sitio público	https://orbital.net.ar
Arrancó	14 de abril de 2026
En producción desde	1 de octubre de 2026
Líneas de código	~30.000
Tests automatizados	414
Costo mensual	US$ 6

2. Cómo empezó, y en qué se convirtió

Abril: dos repositorios y una idea

El proyecto arrancó el **14 de abril** con un commit llamado "*Initial backend split*": separar en dos lo que hasta entonces era una sola cosa. Un backend en Node con Express y un frontend que en ese momento era Next.js.

Los primeros quince días fueron sobre identidad. Integrar Clerk para el login, migrar los usuarios que ya existían, reconciliarlos con las personas de la base. Para el 17 de abril ya había **auditoría automática** —cada operación deja rastro— y control de acceso por rol. Esas dos decisiones tempranas resultaron ser las más importantes de todo el proyecto: todo lo que vino después se apoyó en ellas.

Mayo: el front se rehace, y aparece la segunda persona

El 12 de mayo hay un commit que dice literalmente "*a partir de este commit hacer el front*". Next.js se abandona y el portal se rehace en **React con Vite**. Es la primera vez que el proyecto tira algo a la basura para rehacerlo mejor, y no va a ser la última.

El 13 de mayo aparecen los primeros commits de Gonzalo. De ahí en adelante son dos personas trabajando en paralelo, cada una con su rama.

El 27 de mayo entra algo que cambia el nivel del proyecto: **idempotencia, observabilidad y CI en GitHub Actions**. Que una transferencia no se duplique si el usuario toca dos veces el botón deja de ser una esperanza y pasa a ser una garantía con una clave única en la base.

Junio: el refactor

Junio es el mes menos vistoso y más importante. Casi no hay funcionalidad nueva; hay **modularización**. Los services grandes se parten en pedazos. El SQL se extrae a archivos de queries. Prisma entra como fuente de verdad del esquema —aunque la aplicación nunca lo use en runtime—. En el frontend, `AdminSection` pasa de 520 líneas a 173.

También aparece el **Glosario**: un documento que fija cómo se llama cada cosa. Qué es una `transaccion` y qué es una `transferencia`, qué es `monto` y qué es `importe`, por qué una `cuenta` bancaria no es un `usuario` de acceso. Parece burocracia hasta que dos personas escriben código sobre la misma tabla sin pisarse.

Agosto: la plata deja de ser un número

El 18 de agosto entra el cambio más sutil y más valioso del proyecto: el **value object** `Dinero`, construido sobre `decimal.js`.

Hasta ese día, los montos eran `number` de JavaScript. Y en JavaScript `0.1 + 0.2` no da `0.3`. En un banco eso no es una curiosidad académica: es plata que aparece o desaparece. `Dinero` encapsula el importe, no deja operar entre monedas distintas, y obliga a convertir explícitamente en los bordes del sistema.

En el mismo movimiento se **renombra el dominio entero al español**. `amount` pasa a `monto`, `account` a `cuenta`, `loan` a `prestamo`. El código empieza a hablar el mismo idioma que las conversaciones sobre el código.

En agosto Gonzalo también construye el **chatbot** con Gemini.

Septiembre: el mes en que se convirtió en un banco

Septiembre es, de lejos, el mes más denso. Arranca el 8 con una tanda de commits que cierra **once decisiones** tomadas entre los dos y publica el **contrato OpenAPI** de la API. A partir de ahí el trabajo se organiza en fases, documentadas en `PLAN.md`, con un reparto explícito en `REPARTO.md`:

Fase	Qué trajo
Fase 2	Cuentas multi-moneda y cotización
Fase 3	Extracción, cambio de divisa y movimientos
Fase 4	Tarjetas de débito y crédito

Fase	Qué trajo
Fase 5	Préstamos, plazos fijos y barrido de mora
Fase 6	Proveedores externos (servicios y recargas)
Fase 7	Reportes y asistente con IA

Todo eso en el mismo mes. El 21 de septiembre entra la **sincronización automática de transferencias entrantes**: el Banco Central no avisa cuando llega plata de otro banco, hay que preguntarle. Un proceso de fondo pregunta cada quince minutos, mirando treinta hacia atrás, con ventanas superpuestas a propósito porque un índice único hace que repetir sea seguro.

El 22 de septiembre, el portal entero se vuelve usable en celular.

Octubre: a internet

El 29 de septiembre empieza la **dockerización**. El 1 de octubre, Banco Orbital está en internet: dominio propio, HTTPS con Let's Encrypt, cinco contenedores en un droplet de DigitalOcean de un giga de memoria.

Octubre trae el **rol gerente** con aprobación manual de préstamos, la **sección de administración de roles**, la **landing page**, la **aplicación instalable** en el celular, Cloudflare adelante, los **respaldos de la base**, y los **cobros por QR**.

---

3. Cómo cambió la forma de trabajar

Esta es, para quien vaya a tomar el proyecto, la parte más útil del documento. El código se puede leer; esto no.

De "lo hago y veo si anda" a "lo pruebo y después lo creo"

Durante los primeros meses, verificar significaba abrir el navegador y mirar. Funciona hasta que el sistema tiene veinte pantallas y tocar una rompe otra.

Hoy hay **414 tests automatizados** (386 en el backend, 28 en el portal), pero el número no es lo interesante. Lo interesante es la pregunta que se le hace a cada test nuevo:

> **Si rompo el código a propósito, ¿este test se da cuenta?**

Cada vez que se escribe una tanda de tests, se sabotea el código deliberadamente —se le saca un `trim`, se invierte un booleano, se baja un límite— y se comprueba que los tests fallen. Un test que pasa igual con el código roto no prueba nada; sólo da una falsa sensación de seguridad. Esta práctica se llama *mutation testing* y encontró, entre otras cosas, un error en un arreglo que se acababa de escribir.

De "funciona en mi máquina" a verificar contra lo real

El proyecto dejó de confiar en el razonamiento cuando puede medir:

- Los `nginx.conf` se validan con `nginx -t` sobre las plantillas ya renderizadas y certificados autofirmados, antes de tocar el servidor.
- Las imágenes Docker se construyen y se les pregunta por dentro si los archivos están.
- Las pantallas se fotografían con un navegador real a tamaño de teléfono. Así se descubrió que un marco que se había agregado al escáner de QR quedaba superpuesto con el que la librería ya dibujaba — y se sacó.
- Que una animación efectivamente gire se comprueba sacando dos capturas separadas en el tiempo y verificando que los bytes difieran.
- Los problemas de DNS se diagnostican por capas: registrador, autoritativo, resolver.

De arreglar síntomas a buscar la causa

Hay una diferencia entre "*el alias no funciona*" y "*la interfaz le ofrece al cliente un botón que pega a un endpoint que sólo el personal del banco puede usar*". Los commits del proyecto cuentan la segunda historia, no la primera.

El hábito es: antes de tocar nada, reproducir. Antes de afirmar, verificar. Y cuando una hipótesis resulta falsa, decirlo. Hubo un momento en que se dio por hecho que `pg_dump` no funcionaba contra el pooler de Supabase en modo transacción; se probó, funcionaba, y el comentario del script se corrigió antes de subirlo. La documentación que miente es peor que la que no existe.

Los comentarios explican por qué, no qué

El código del proyecto casi no tiene comentarios que digan lo que el código ya dice. Tiene comentarios que explican **la decisión** y, muy seguido, el error que la motivó:

> "*En Express 5 `req.query` es una propiedad de sólo lectura. Asignarle no lanza error, pero tampoco hace nada, y el handler sigue recibiendo los strings crudos. Eso dejaba sin efecto toda la paginación: pedir `?limit=100` devolvía 20 igual.*"

Ese comentario le ahorra media hora a quien lo lea en marzo.

Los mensajes de commit cuentan qué se rompía

Un commit del proyecto no dice "*fix: cors*". Dice qué estaba mal, por qué importaba, y cómo se comprobó que quedó arreglado. El historial de Git funciona como la bitácora del proyecto.

Trabajar de a dos sin pisarse

Cada persona trabaja en su rama (`masita` y `gonza`), ambas se integran a `main`. El reparto quedó escrito en `REPARTO.md` para que nadie tenga que adivinar de quién es cada cosa, y las

once decisiones de arquitectura se discutieron y se anotaron antes de escribir el código que dependía de ellas.

Cuando una persona revisa el trabajo de la otra, lo lee entero. La revisión del módulo de cobros por QR —352 líneas— encontró que el pago no validaba el límite de transferencia de la cuenta: por QR se movía el doble de lo que el banco permitía por el formulario de siempre. Era un módulo bien escrito, con bloqueos correctos y auditoría completa. El error estaba en lo que faltaba, no en lo que estaba.

---

4. Arquitectura

Los cinco repositorios

Repositorio	Qué es	Tecnología
`banco-backend`	La API y toda la lógica del banco	Node 22, Express 5, PostgreSQL
`banco-frontend`	El home banking	React 18, Vite 6, Tailwind v4
`banco-landing`	El sitio público	React 18, Vite 6, Tailwind v4
`banco-proveedores`	Mock de servicios externos	Node, Express
`banco-infra`	Contenedores, nginx, despliegue	Docker Compose, nginx, certbot

Cómo se conectan en producción

```
internet ──→ Cloudflare ──→ :443 nginx (el único expuesto)
                            ├── app.orbital.net.ar
                            │     ├── /            → portal (React compilado)
                            │     ├── /api  /auth  → backend:3001
                            │     └── /webhooks/   → backend:3001  (Clerk)
                            └── orbital.net.ar     → landing

backend ──→ proveedores:4000      (red interna, no sale a internet)
backend ──→ Supabase              (PostgreSQL 17)
backend ──→ Banco Central         (transferencias interbancarias)
backend ──→ DolarAPI              (cotización)
backend ──→ Gemini                (el asistente)
```

El portal y la API viven **en el mismo origen**. Eso elimina el CORS del navegador y hace que el token de Clerk viaje sin fricción.

La base de datos

PostgreSQL 17 en Supabase, **22 tablas**. No corre en el servidor: es un servicio aparte, y el droplet sólo ejecuta la aplicación. Las migraciones no las aplica ningún contenedor; se corren

a mano con `psql`. Es la forma más difícil de romper la base por accidente.

Las tablas centrales:

Tabla	Para qué
`personas`	La persona física: nombre, DNI, correo
`usuarios`	La identidad de acceso, atada a Clerk
`cuentas`	Las cuentas bancarias, con CBU, alias, moneda y saldo
`transacciones`	Todo movimiento de plata
`tarjetas`	Débito y crédito, con su nivel
`prestamos` / `cuotas_prestamo`	Préstamos y su plan de pagos
`plazos_fijos`	Inversiones a plazo
`cobros`	Los QR de cobro
`auditoria`	Quién hizo qué, cuándo y desde qué IP
`idempotency_keys`	Para que una operación no se ejecute dos veces

---

5. Decisiones técnicas y por qué

`Dinero`: la plata no es un `number`

`0.1 + 0.2 !== 0.3` en JavaScript. En un banco eso es plata que aparece o desaparece. Todo importe se maneja con un value object sobre `decimal.js` que además **impide operar entre monedas distintas** —no se puede sumar un peso a un dólar sin convertir explícitamente— y se convierte a número sólo en el borde, al responder la API.

Idempotencia: tocar dos veces el botón no cobra dos veces

Toda operación que mueve plata exige una `Idempotency-Key`. Si llega repetida, la respuesta original se devuelve tal cual, sin volver a ejecutar nada. Es la diferencia entre un sistema que se puede usar con mala señal y uno que no.

Bloqueos en orden fijo

Cuando una operación toca dos cuentas, las bloquea con `SELECT ... FOR UPDATE` **ordenadas por id**. Sin eso, dos transferencias cruzadas simultáneas se bloquean mutuamente para siempre. El débito, además, se hace en una sola sentencia:

```
UPDATE cuentas SET saldo = saldo - $1 WHERE id = $2 AND saldo >= $1
```

Si devuelve cero filas, no había saldo. No hay ventana entre comprobar y descontar.

Auditoría que no se puede esquivar

Cada operación deja un registro con usuario, acción, entidad, el antes y el después, la IP y el origen. Es append-only: el rol `auditor` puede leerla y nadie la puede modificar.

`node:22-slim`, no `alpine`

El formateo de plata y fechas usa `Intl` con locale `es-AR`. Alpine trae un ICU recortado y eso devuelve strings distintos **en silencio** — que es la peor forma de fallar.

La zona horaria del contenedor

Un préstamo pedido a las 22:00 de Buenos Aires aparecía fechado al día siguiente, porque el contenedor corría en UTC. Se arregla con una variable de entorno, pero encontrarlo costó.

`trust proxy`

Detrás de nginx, sin esta línea el rate limit cuenta a todos los usuarios como una sola IP y la auditoría registra la IP interna de Docker en vez de la de quien operó. Es una línea de código y, de no estar, invalida dos sistemas de seguridad a la vez.

---

6. Qué sabe hacer el banco

Para el cliente

- **Cuentas** en pesos y en dólares, con CBU y alias editable.
- **Transferencias** a cuentas del mismo banco y a otros bancos vía el Banco Central, con comprobante en PDF.
- **Dólares**: compra y venta con cotización en vivo, y un botón para operar con todo el saldo.
- **Tarjetas** de débito y crédito. Las de crédito tienen cuatro niveles —Standard, Gold, Platinum y Black— que se otorgan según la situación crediticia.
- **Préstamos** con consulta automática a la Central de Deudores. Si la situación es buena, se acredita solo; si no, pasa a revisión de un gerente.
- **Plazos fijos** con cálculo de interés y barrido de vencimientos.
- **Servicios y recargas** contra el mock de proveedores.
- **Cobros por QR**: generar un código temporal de diez minutos y pagarlo escaneándolo.
- **Asistente con IA** que responde sobre saldos, movimientos y ofertas de crédito reales.
- **Aplicación instalable** en el celular, sin pasar por ninguna tienda.

Para el personal del banco

Rol	Qué puede hacer
`cliente`	Operar con lo suyo, y nada más
`operador`	Alta y gestión de clientes y cuentas
`tesoreria`	Operaciones internas de caja
`gerente`	Aprobar o rechazar préstamos que el filtro automático no resolvió
`auditor`	Leer la auditoría; no puede modificar nada
`admin`	Todo, incluido repartir roles

El gerente es el único que puede aprobar un préstamo a alguien con mala situación crediticia, y para hacerlo **tiene que escribir un motivo de al menos diez caracteres** que queda registrado con su nombre.

---

7. Cómo se corre

En tu máquina

```
# Backend
cd banco-backend && npm install && npm run dev      # :3001

# Proveedores
cd banco-proveedores && npm install && npm run dev  # :4000

# Portal
cd banco-frontend && npm install && npm run dev     # :5173
```

Hace falta un `.env` en cada uno. Los `.env.example` listan las variables.

> **Node 20 o superior.** Con Node 18 el plugin de PWA falla con `crypto is not defined`.

Los tests

```
cd banco-backend  && npx vitest run    # 386
cd banco-frontend && npx vitest run    # 28
```

Desplegar

```
ssh maxi@<ip-del-droplet>
cd /opt/orbital/banco-infra
./scripts/deploy.sh
```

Trae los cambios de los cinco repositorios, reconstruye lo que haga falta y limpia las capas viejas de Docker. El paso de limpieza no es opcional: el disco se llena en pocos deploys.

> **El build ahoga la máquina.** Compilar el portal en un giga de memoria consume todo mientras dura. Si justo después de deployar no entra por SSH, no es la clave ni el servidor caído: hay que esperar dos minutos.

Respaldar la base

```
./scripts/respaldo.sh            # un dump comprimido
./scripts/respaldo.sh --listar   # qué hay guardado
```

> **El snapshot de DigitalOcean no respalda el banco.** Respalda el droplet: nginx, los `.env`, los contenedores. Los datos están en Supabase, que es otra máquina de otra empresa. Son dos cosas distintas y conviene tener las dos.

---

8. Errores que costaron encontrar

Esta lista existe para que nadie los vuelva a sufrir.

Síntoma	Causa real
`?limit=100` devolvía 20 filas	En Express 5 `req.query` es de sólo lectura. Asignarle no falla, pero no hace nada. Hay que redefinir la propiedad.
Préstamos fechados al día siguiente	El contenedor corría en UTC y el código usaba hora local.
El rate limit afectaba a todos juntos	Faltaba `trust proxy`: nginx hacía que todos parecieran la misma IP.
Un QR movía el doble del límite permitido	El pago por QR no consultaba el `limite_transferencia` de la cuenta.
El aviso de "préstamo aprobado" no aparecía	La tarjeta se desmonta al resolver, y React Query descarta los callbacks de un componente que ya no existe. La mutación tenía que vivir en la sección.
Flash blanco al abrir la app instalada	El `background_color` del manifest era blanco y el portal es oscuro.

Síntoma	Causa real
Certificado de prueba quedó sirviendo	El ensayo y el real usaban el mismo nombre de certificado.
El script de certificados no arrancaba	Un apóstrofo dentro de un `${VAR:?...}` en bash.
El portal local quedó bloqueado por CORS	El default de `CORS_ORIGINS` pasó a ser el dominio de producción, sin avisar.
No se podía entrar por SSH	El build del portal consumía toda la memoria; `sshd` aceptaba la clave pero no podía abrir la sesión.

9. Qué queda por hacer

Tarea	Por qué
Monitoreo de uptime	Si el servidor se cae a las 3 AM, hoy nadie se entera
Cron del respaldo	El script está; falta la línea que lo corra solo
Clerk en modo producción	Saca el cartel de "development" y los límites de usuarios
Rotar la clave de Gemini	Viajó por un canal no seguro
Probar el pago por QR de punta a punta	Nunca se ejecutó en producción: la auditoría registra un QR generado y ningún pago
Borrar `personas_backup_clerk_20260416`	Tabla de respaldo de abril que quedó dando vueltas

10. Dónde está cada cosa

Documento	Qué hay adentro
`banco-infra/README.md`	Montar el servidor de cero, el día a día, respaldos
`banco-infra/docs/manual-de-despliegue.pdf`	El despliegue paso a paso, ilustrado
`banco-backend/README.md`	Setup, endpoints, arquitectura de la API
`banco-backend/docs/GLOSARIO.md`	Cómo se llama cada cosa y por qué
`banco-backend/docs/PLAN.md`	Las fases y las decisiones tomadas

Documento	Qué hay adentro
`banco-backend/docs/REPARTO.md`	Quién hizo qué
`banco-backend/docs/openapi-banco-orbital.yaml`	El contrato de la API

Banco Orbital — Máximo Bertaina y Gonzalo Castellano · 2026